Práctica de la Unidad II

Estructuras de datos básicas

LISTAS PILAS COLAS

12 ejercicios / 3 proyectos de escritorio / Java 21

Institución	Universidad Tecnológica Metropolitana
División	Tecnologías de la Información y Comunicación
Carrera	Ingeniería en Tecnologías de la Información e Innovación Digital
Asignatura	Estructura de Datos
Docente	Ing. Mirian Magaly Canché Caamal
Integrantes	Pendiente de completar por el equipo
Grupo	Pendiente de completar
Entrega	23 de octubre de 2026
Elaboración	16 de octubre de 2026

Implementación orientada a objetos con menús funcionales, validación de entradas y documentación de resultados.

Datos por completar: integrantes y grupo. No se inventaron identidades ni un grupo del equipo.

Guía de ejecución y contenido

Objetivo. Implementar las operaciones de listas, pilas y colas y comprobar su comportamiento mediante interfaces de escritorio y pruebas reproducibles.

Requisitos. Windows con Java 21 o posterior. La aplicación usa únicamente la biblioteca estándar: Swing, colecciones, BigDecimal y java.time. No requiere Internet. Para recompilar se necesita un JDK.

Inicio. Abrir INICIAR.cmd para acceder al menú general, o el INICIAR.cmd de proyectos/Listas, proyectos/Pilas o proyectos/Colas para trabajar con cada estructura por separado.

Navegación. Elegir una opción en el menú lateral. Los módulos conservan sus datos mientras la aplicación permanece abierta. Cada acción valida los campos y permite continuar sin reiniciar todo el programa. Salir cierra el proyecto.

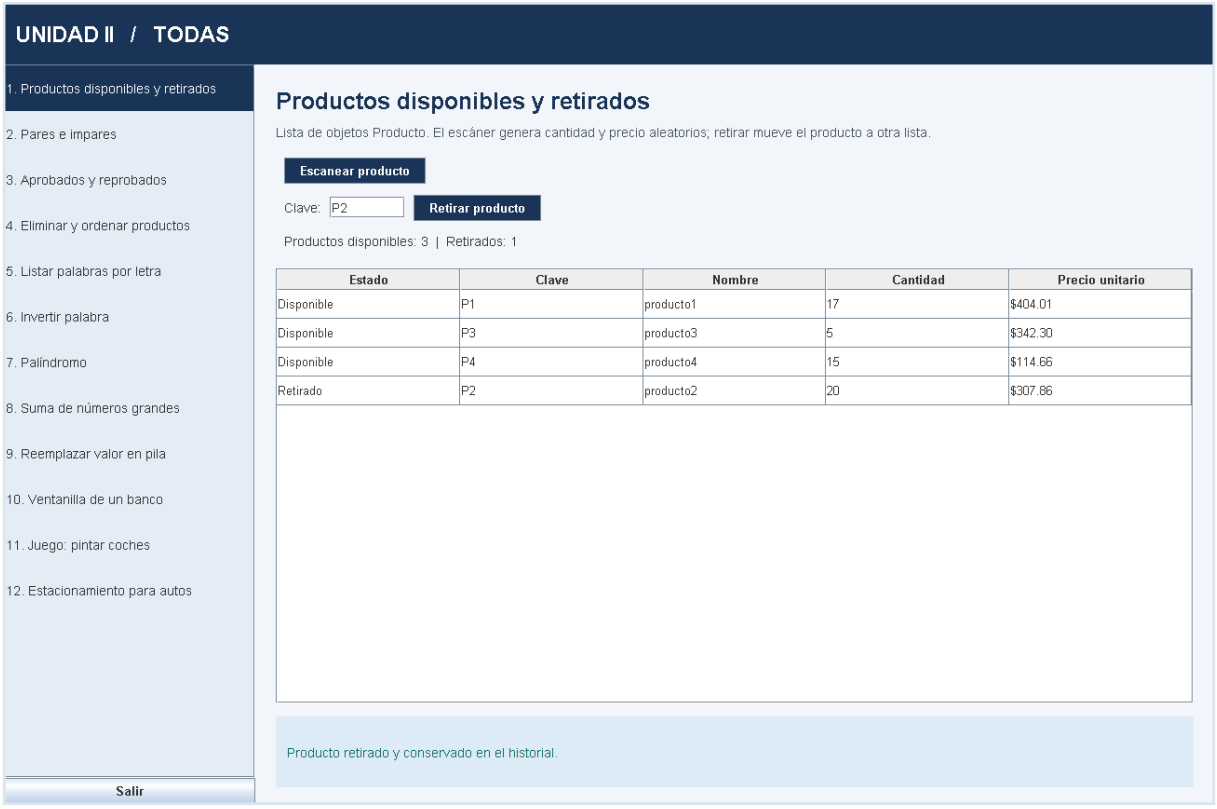


Figura 1. Menú general y primer ejercicio después de ejecutar las operaciones de demostración. Los tres proyectos independientes ofrecen sólo las opciones de su estructura.

Listas	L1-L5: productos, pares/impares, alumnos, compra, palabras.
Pilas	P1-P4: inversión, palíndromo, suma grande, reemplazo.
Colas	C1-C3: banco, juego de coches, estacionamiento.
Verificación	Evidencias ampliadas, validaciones, resultados de pruebas y trazabilidad.

Arquitectura y decisiones

Modelo	Estructuras.java: objetos Producto, Alumno, Cliente, Coche y Auto; operaciones de cada estructura sin dependencias de Swing.
Interfaz	PracticaApp.java: un panel por ejercicio, campos con nombres descriptivos, botones, tablas no editables y mensajes de error o confirmación.
Proyectos	Listas, Pilas y Colas incluyen Inicio.java, fuentes, clases y JAR. Cada Inicio abre su menú correspondiente.
Pruebas	Pruebas.java: validaciones de algoritmos, límites, FIFO/LIFO, enlaces, tiempos e integración de botones. Produce registro y evidencias.
Recursos	Ilustraciones originales de coches dibujadas con Graphics2D; sin imágenes remotas ni dependencias adicionales.

Estructuras empleadas. ArrayList almacena listas. ArrayDeque se usa como pila mediante push/pop y como base de la cola FIFO acotada. El estacionamiento no usa una lista como sustituto de sus enlaces: implementa nodos con anterior y siguiente, cerrados circularmente.

Entradas numéricas. Precios y calificaciones aceptan decimales con punto o coma. Los importes se manejan con BigDecimal y se muestran con dos decimales. Cantidades, turnos y valores de pila requieren enteros. La suma grande recibe cadenas de dígitos no negativos.

Reglas no especificadas. Se fija capacidad inicial del banco en 5, modificable entre 1 y 1000; generación aleatoria de 0 a 999 y máximo de 500 números. En el juego se reduce el intervalo 2 s cada 3 aciertos, con mínimo 2 s y primer coche inmediato.

Tiempo y datos. Las operaciones normales usan la hora del sistema. El estacionamiento cobra segundos completos a $2.00 cada uno. Los datos permanecen durante la sesión. El récord del juego se guarda en la carpeta de usuario .practica-u2/record.properties.

Interpretación del instrumento. En listas 3, la repetición de “aprobaron” se resuelve mostrando ambas listas. En pilas 4 se entiende viejo como el valor buscado y nuevo como su sustituto. Se elige Java Swing porque el lenguaje y el tipo de aplicación son libres.

L1. Productos disponibles y retirados

Problema. Almacenar objetos Producto escaneados con cantidad y precio aleatorios, permitir su retiro e informar cuántos productos están disponibles y cuántos fueron retirados.

Entradas	Acción Escanear; clave del producto que se desea retirar.
Proceso	Crear Producto con clave secuencial, nombre productoN, cantidad 1 a 20 y precio $1.00 a $500.00. Insertarlo en ArrayList; al retirarlo, moverlo a otra lista.
Salidas	Tabla con ambos estados, cantidad y precio unitario; contadores de objetos disponibles y retirados.

Funcionamiento paso a paso. 1. Escanear cuatro productos. 2. Escribir P2 en Clave. 3. Pulsar Retirar producto. 4. Revisar los contadores y el historial.



Figura L1. Se ejecutaron cuatro escaneos y un retiro mediante los botones. Quedan tres objetos disponibles y uno retirado; producto2 conserva sus datos en el historial. Los valores aleatorios pueden variar.

Pruebas y validaciones. Se verificaron 100 altas, rangos aleatorios, traslado a retirados y rechazo de un segundo retiro de la misma clave.

L2. Pares e impares

Problema. Generar una lista de números aleatorios y separar los valores pares e impares en dos listas adicionales.

Entradas	Cantidad entera de elementos, entre 1 y 500.
Proceso	Generar enteros de 0 a 999. Recorrer la lista original y aplicar número % 2: resto cero se agrega a pares, los demás a impares.
Salidas	Lista original, lista de pares y lista de impares, cada una con su número de elementos.

Funcionamiento paso a paso. 1. Introducir 15 en Cantidad. 2. Pulsar Generar y clasificar. 3. Comparar las tres listas; repetir sin cerrar la aplicación.



Figura L2. El botón produjo una lista de 15 números y mostró su distribución. La suma de las cantidades de pares e impares coincide con el tamaño original; se conservan los repetidos.

Pruebas y validaciones. Con semilla fija se generaron 500 valores y se verificó su paridad y la conservación de todas sus multiplicidades. Se rechaza cantidad cero.

L3. Aprobados y reprobados

Problema. Registrar alumnos con sus calificaciones y extraer de la lista inicial a quienes tengan menos de siete para almacenarlos en reprobados.

Entradas	Nombre no vacío y calificación decimal de 0 a 10.
Proceso	Agregar el objeto Alumno a la lista inicial. Recorrerla con Iterator y trasladar a reprobados los registros con nota menor que 7.
Salidas	Tabla que identifica la lista de cada alumno, su nota y las cantidades de aprobados y reprobados.

Funcionamiento paso a paso. 1. Registrar Ana López con 9.5. 2. Registrar Luis Pérez con 6.9. 3. Registrar María Díaz con 7. 4. Revisar ambas listas.



Figura L3. Ana y María aparecen en aprobados; Luis fue trasladado a reprobados. La nota 7 se considera aprobatoria y admite valores decimales.

Pruebas y validaciones. Se comprobaron notas 0, 6.99, 7 y 10. Se rechazan 11 y el nombre vacío. Se interpreta la repetición de “aprobaron” del enunciado como mostrar aprobados y reprobados.

L4. Eliminar y ordenar productos

Problema. Agregar productos con clave, nombre y precio, eliminar por clave, mostrar los nombres ordenados y calcular el costo total de compra.

Entradas	Clave única, nombre y precio decimal no negativo; clave a eliminar.
Proceso	Guardar Producto en ArrayList. Eliminar la coincidencia por clave. Ordenar una copia por nombre y sumar los precios con BigDecimal.
Salidas	Tabla alfabética y total monetario actualizado después de cada operación.

Funcionamiento paso a paso. 1. Agregar Zanahoria ($15.50), Arroz ($32.75), Leche ($26.25) y Pan ($20.00). 2. Eliminar P4 (Pan). 3. Revisar orden y total.

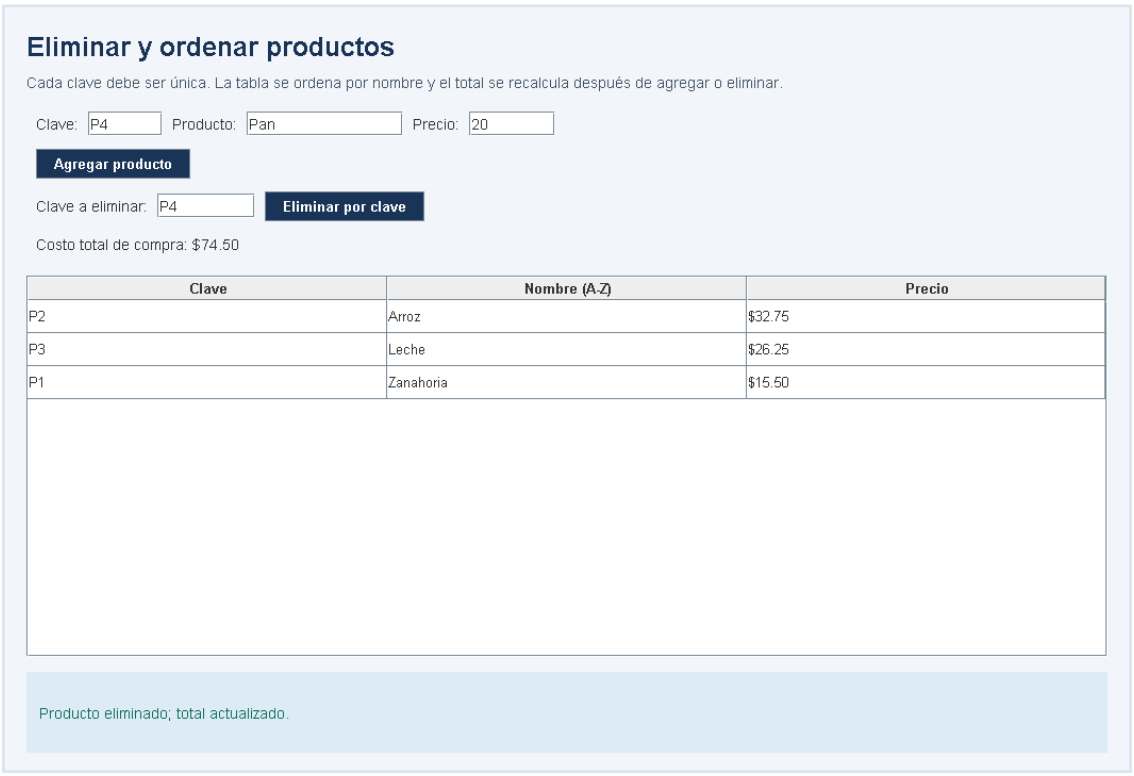


Figura L4. Después de eliminar P4, se muestran Arroz, Leche y Zanahoria. El total correcto es $74.50. Los importes se calculan sin errores de punto flotante.

Pruebas y validaciones. Se verificó orden por nombre, total $74.50, eliminación y recálculo; se rechazan claves duplicadas, inexistentes y precios negativos.

L5. Listar palabras por letra

Problema. Almacenar palabras y distribuirlas en listas según su primera letra, creando una nueva lista para cada inicial diferente.

Entradas	Una palabra por registro, formada exclusivamente por letras; acepta acentos.
Proceso	Conservar ArrayList original. Obtener la primera letra en mayúsculas y agrupar en un mapa ordenado cuyas entradas contienen listas independientes.
Salidas	Lista original y una lista rotulada por cada inicial encontrada.

Funcionamiento paso a paso. 1. Agregar Fresa y Fresca. 2. Agregar Orilla, Oso, Peso y Pez. 3. Agregar Árbol. 4. Revisar la nueva lista de Á.



Figura L5. Las palabras del ejemplo quedan reunidas en F, O y P. Al registrar Árbol se crea automáticamente una cuarta lista con la inicial Á.

Pruebas y validaciones. Se comprobó la agrupación sin distinguir mayúsculas, la inicial acentuada y el rechazo de una cadena con varias palabras.

P1. Invertir palabra

Problema. Leer una palabra e invertir sus caracteres mediante una pila, como UTM que produce MTU.

Entradas	Palabra o texto no vacío.
Proceso	Recorrer los puntos de código Unicode e insertarlos con push. Extraer cada uno con pop y concatenarlo al resultado hasta vaciar la pila.
Salidas	Entrada original y secuencia invertida.

Funcionamiento paso a paso. 1. Escribir UTM en Palabra. 2. Pulsar Invertir. 3. Leer la salida MTU. 4. Introducir otra palabra y repetir.



Figura P1. La pantalla muestra UTM y MTU. El resultado se obtiene por la extracción LIFO de los caracteres, sin usar reverse como sustituto de la pila.

Pruebas y validaciones. Se validaron UTM y una cadena con un carácter Unicode fuera del plano básico. Una entrada vacía produce un mensaje de datos faltantes.

P2. Palíndromo

Problema. Determinar si una palabra, número o frase se lee igual de izquierda a derecha que en sentido inverso.

Entradas	Texto con al menos una letra o un dígito.
Proceso	Pasar a minúsculas y conservar letras y dígitos. Invertir la secuencia normalizada mediante una pila y comparar ambas cadenas.
Salidas	Mensaje afirmativo o negativo sobre si es un palíndromo.

Funcionamiento paso a paso. 1. Introducir Anita lava la tina. 2. Pulsar Comprobar palíndromo. 3. Revisar el resultado. 4. Repetir con Casa para un caso negativo.

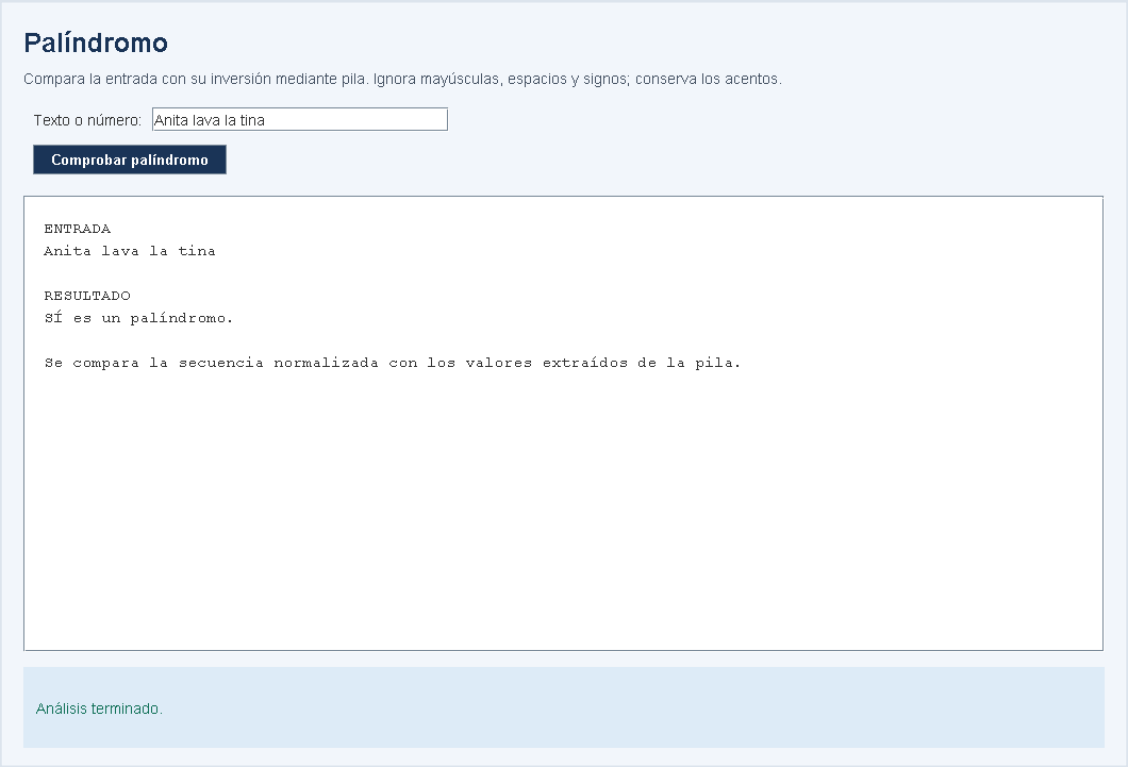


Figura P2. Anita lava la tina se identifica como palíndromo al ignorar espacios y mayúsculas. Los acentos se conservan y no se eliminan de la comparación.

Pruebas y validaciones. Se verificaron una frase, 12321 y Casa. Una entrada compuesta sólo por signos se rechaza porque carece de letras y números.

P3. Suma de números grandes

Problema. Sumar enteros muy grandes representados como cadenas usando pilas de dígitos, evitando el límite de los tipos enteros básicos.

Entradas	Dos cadenas de dígitos del 0 al 9, sin signos ni separadores; se consideran enteros no negativos.
Proceso	Apilar cada operando. Extraer unidades, sumar los dígitos y el acarreo, apilar el dígito resultante y repetir. Extraer la tercera pila para formar la suma.
Salidas	Suma exacta como cadena, sin ceros iniciales innecesarios.

Funcionamiento paso a paso. 1. Introducir treinta dígitos 9 en el primer campo. 2. Introducir 1 en el segundo. 3. Pulsar Sumar con pilas y revisar el acarreo final.

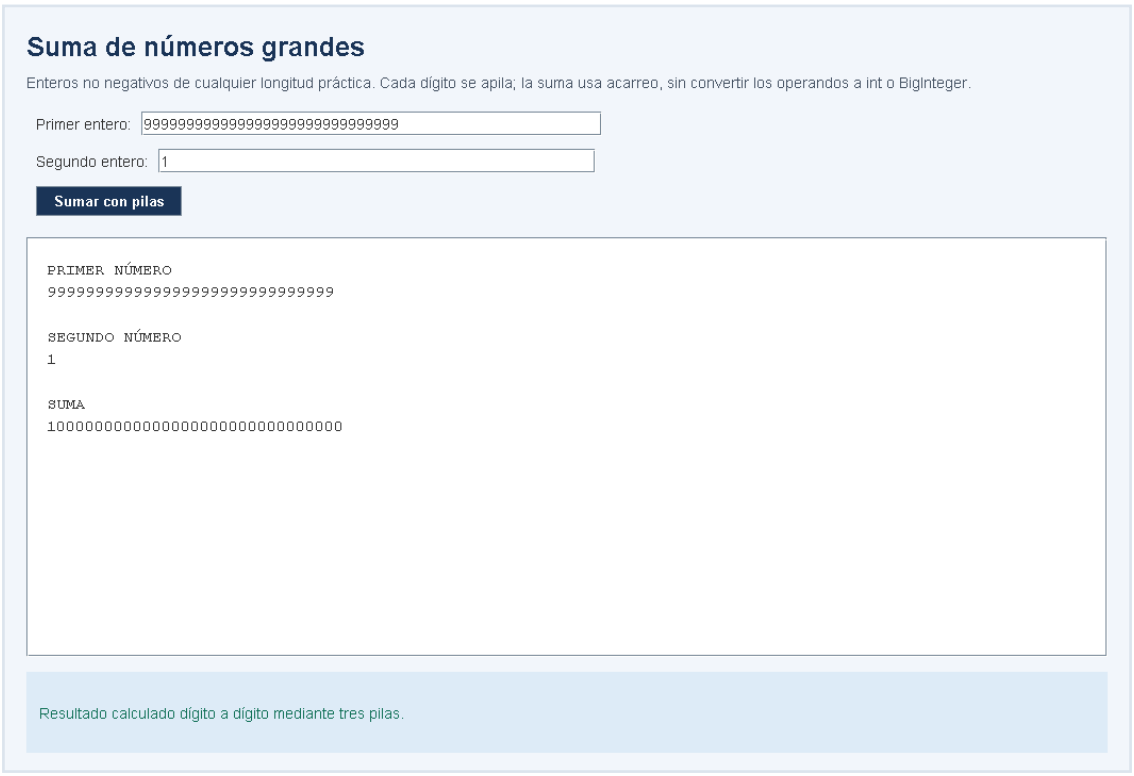


Figura P3. La suma de treinta nueves y uno produce un 1 seguido de treinta ceros. Los operandos no se convierten a int, long ni BigInteger en la solución.

Pruebas y validaciones. Se compararon 200 sumas aleatorias de hasta 1234 dígitos con BigInteger como oráculo exclusivo de pruebas. También se verificaron ceros iniciales y entradas inválidas.

P4. Reemplazar valor en pila

Problema. Implementar Reemplazar(pila, nuevo, viejo) para sustituir todas las apariciones del valor viejo por el nuevo sin alterar el orden de la pila.

Entradas	Enteros para apilar; valores enteros viejo y nuevo, dentro del rango de int.
Proceso	Desapilar en una pila auxiliar, sustituyendo las coincidencias y contando cambios. Devolver los elementos a la pila original para restaurar su orden.
Salidas	Pila mostrada de cima a base y cantidad de reemplazos realizados.

Funcionamiento paso a paso. 1. Apilar 1, 2, 3 y 2 en ese orden. 2. Introducir viejo=2 y nuevo=9. 3. Pulsar Reemplazar. 4. Revisar cima y base.



Figura P4. Antes, la pila de cima a base era [2, 3, 2, 1]. Después queda [9, 3, 9, 1], con dos reemplazos. La posición relativa de los otros elementos se conserva.

Pruebas y validaciones. Se comprobó la sustitución múltiple, valor ausente y pila vacía. Se interpreta el error de redacción del instrumento como reemplazar viejo por nuevo.

C1. Ventanilla de un banco

Problema. Administrar una cola de clientes con turno, nombre, movimiento y hora de llegada; atender al primero y calcular su tiempo de espera.

Entradas	Capacidad (inicial 5), nombre y movimiento seleccionado en combo; turno y llegada se obtienen automáticamente.
Proceso	InsertaCola agrega al final si ColaLlena es falso. EliminaCola atiende el frente si ColaVacia es falso y resta la llegada a la hora actual del sistema.
Salidas	Tabla actualizada, frente/final, confirmación del alta y aviso con el cliente atendido y espera en segundos.

Funcionamiento paso a paso. 1. Registrar Ana, Luis y María. 2. Seleccionar el movimiento antes de cada alta. 3. Pulsar Atender en ventanilla. 4. Revisar los turnos restantes.

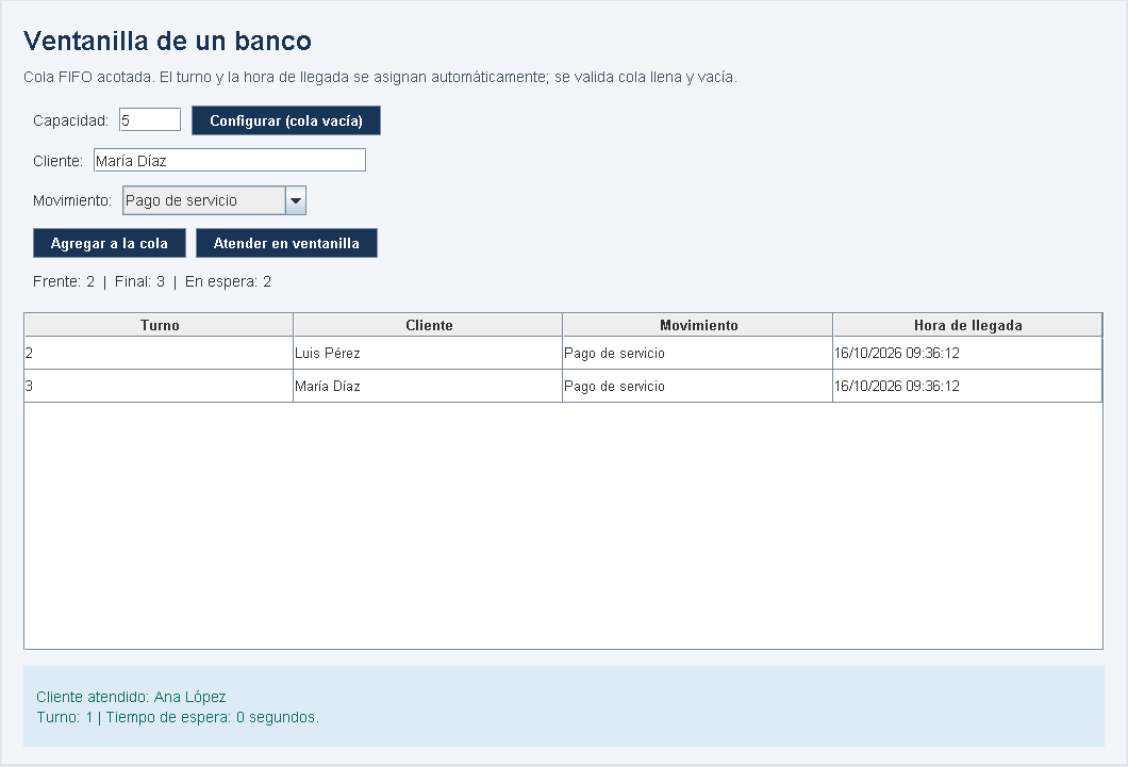


Figura C1. Ana, turno 1, fue atendida primero. Luis y María permanecen en orden; frente=2, final=3. El mensaje muestra la espera medida por el sistema durante esta ejecución.

Pruebas y validaciones. Se verificaron FIFO, sobreflujo, cola vacía y espera exacta de 18 segundos con instantes controlados. La capacidad se modifica sólo con la cola vacía.

C2. Juego: pintar coches

Problema. Pintar el primer coche de una cola con el color solicitado antes de que se acumulen cinco coches; mostrar coches pintados, duración y récord.

Entradas	Botón Iniciar y selección de color en la paleta.
Proceso	Encolar un coche inicial y otro cada 20 s con color aleatorio. Validar el color del frente; al acertar, desencolar. Cada tres aciertos reducir el intervalo 2 s hasta 2 s.
Salidas	Coches ilustrados, etiquetas de color, siguiente llegada, tiempo, puntaje, récord y fin de partida al alcanzar cinco.

Funcionamiento paso a paso. 1. Pulsar Iniciar. 2. Leer la etiqueta del primer coche. 3. Seleccionar ese color en la paleta. 4. Continuar mientras llegan nuevos coches.

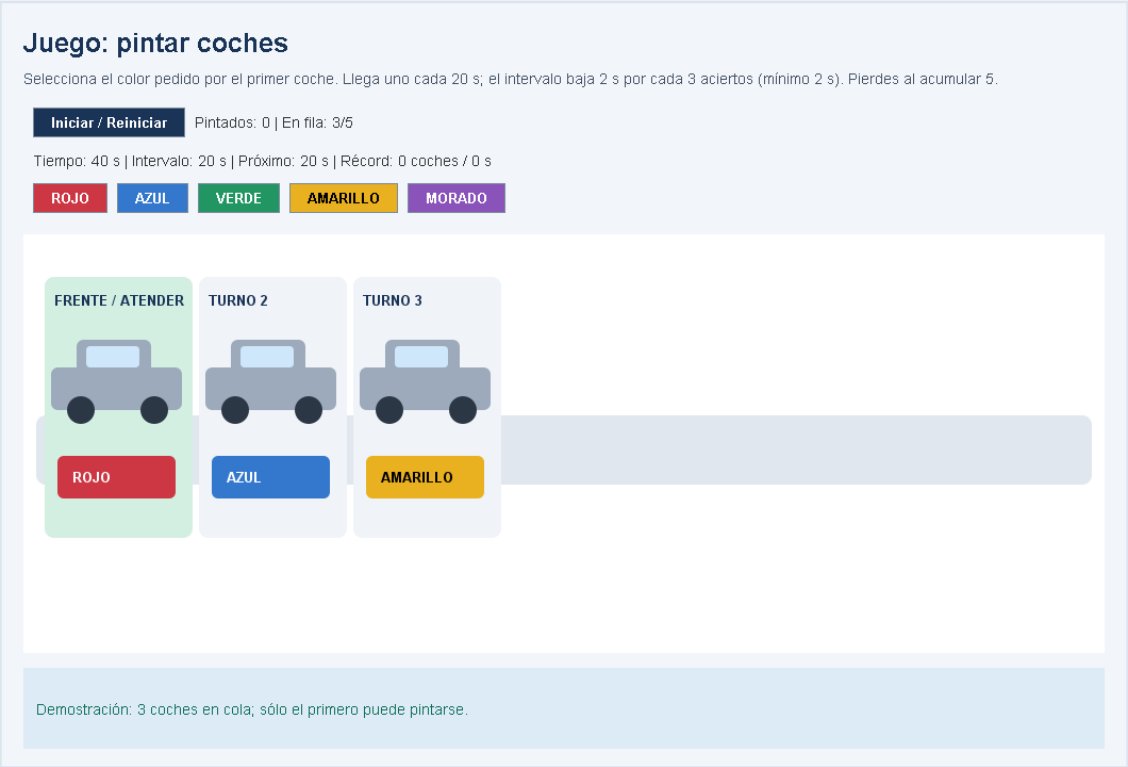


Figura C2. La captura corresponde a 40 segundos simulados en el modelo real: hay tres coches. El frente se identifica en verde y sólo su color puede retirar un coche de la cola.

Pruebas y validaciones. Se comprobaron llegada a los 20 s, color equivocado, aceleración, intervalo mínimo, reinicio y fin exacto con cinco coches. El récord se guarda localmente.

C3. Estacionamiento para autos

Problema. Administrar un estacionamiento sin cupo fijo mediante una cola circular doblemente ligada, con entrada por el final y salida por el frente; cobrar $2.00 por segundo.

Entradas	Placas y propietario no vacíos; fecha y hora de entrada tomadas del sistema.
Proceso	Crear un nodo Auto y enlazarlo con el último y el primero. Para salir, quitar el nodo frente y reparar ambos enlaces. Multiplicar los segundos completos por $2.00.
Salidas	Confirmación de entrada, tabla FIFO y recibo con placas, propietario, entrada, salida, tiempo y costo.

Funcionamiento paso a paso. 1. Registrar ABC-123 y XYZ-987. 2. Dar salida al primer auto. 3. Revisar su recibo. 4. Agregar MEX-456 y verificar los enlaces.

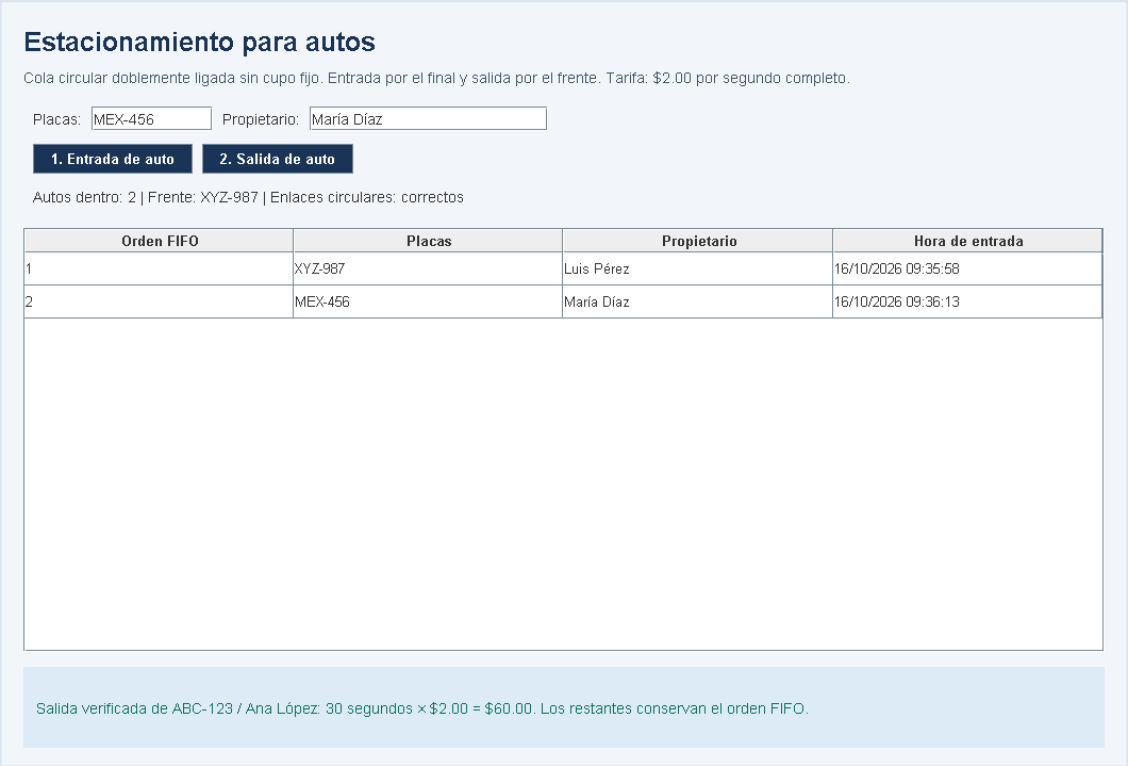


Figura C3. Se retiró ABC-123 tras 30 segundos de prueba, por $60.00. Los autos restantes son XYZ-987 y MEX-456 en ese orden. La pantalla confirma que los enlaces circulares son correctos.

Pruebas y validaciones. Se comprobaron salida en vacío, placas duplicadas, caso de un solo nodo, cobro y mil inserciones/salidas sin cupo fijo, validando los enlaces en cada eliminación.

Juego: aceleración y fin de partida

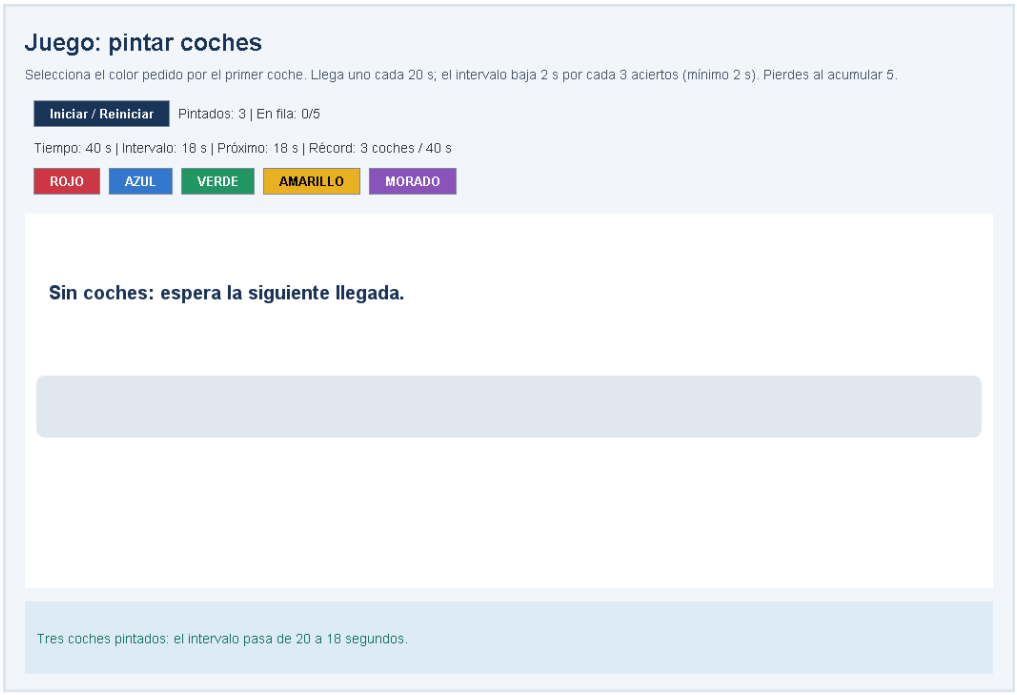


Figura C2-A. Después de tres aciertos, el contador indica tres coches pintados y el intervalo disminuye de 20 a 18 segundos. El modelo avanza con un reloj controlado para probar el cambio sin esperas reales.

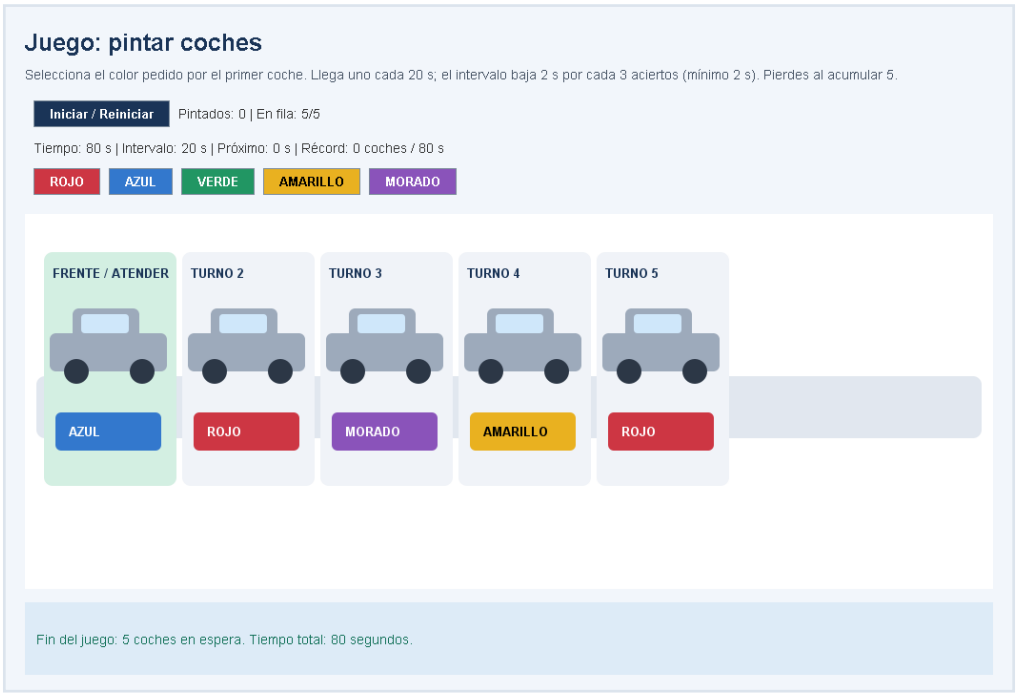


Figura C2-B. Sin pintar coches, se acumulan cinco al segundo 80 (uno inicial y cuatro llegadas). El juego detiene su tiempo y conserva el resultado. No permite retirar coches hasta iniciar otra partida.

Durante la ejecución interactiva, un Timer de Swing revisa el reloj cada 100 ms. El modelo decide las llegadas con marcas de tiempo; si hay un retraso, procesa las llegadas pendientes en orden.

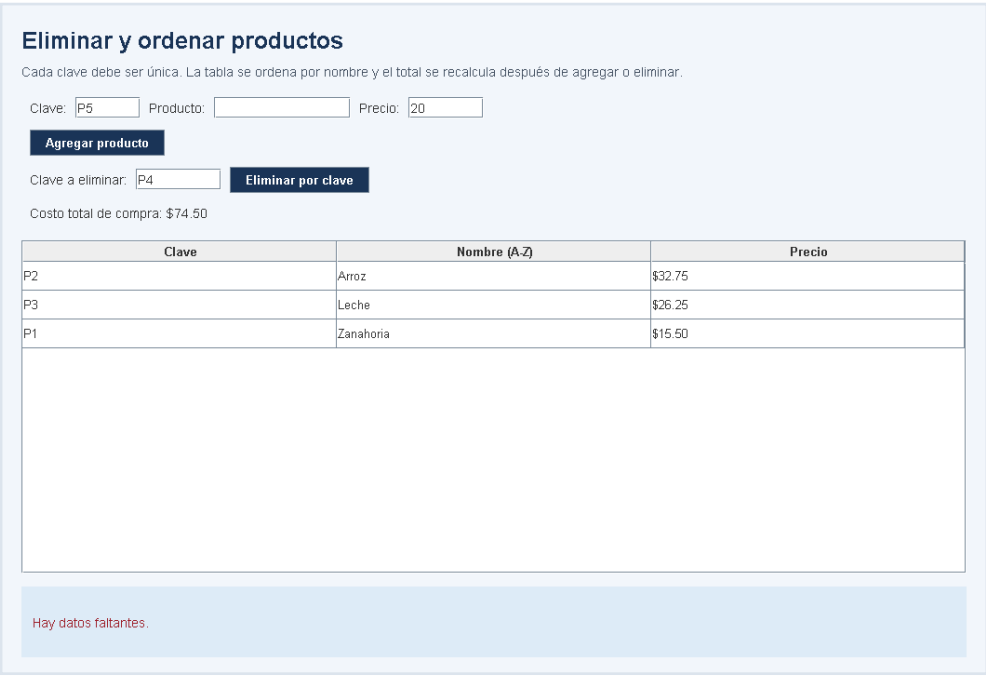
Validaciones de entrada y operación

Entrada no numérica



Al introducir 12x en la suma, se muestra un mensaje que solicita enteros no negativos. La aplicación permanece abierta y permite corregir la entrada.

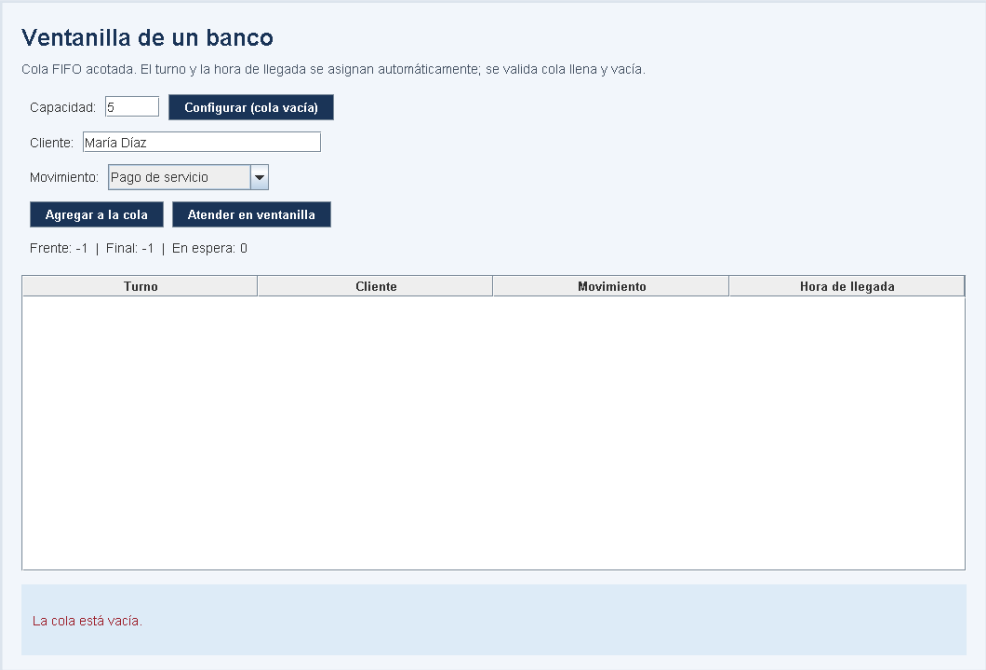
Datos faltantes



Al intentar registrar un producto sin nombre se muestra “Hay datos faltantes”. La lista y el total se conservan sin agregar un registro incompleto.

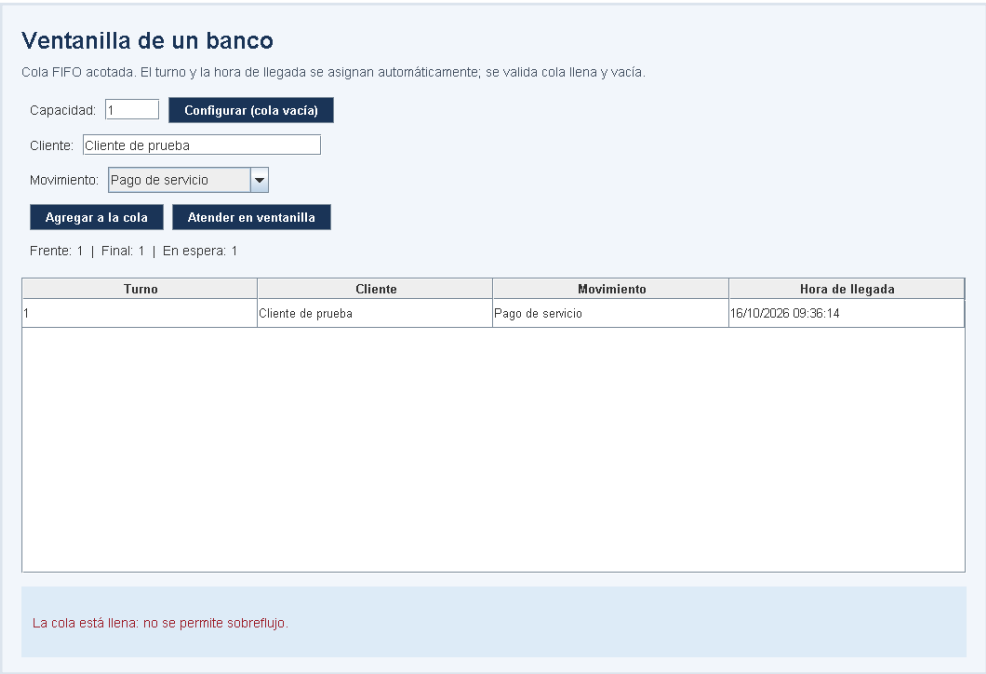
Validaciones de entrada y operación

Cola vacía



Después de atender a todos los clientes, una nueva solicitud de atención informa que la cola está vacía. Frente y final se muestran como -1.

Cola llena



Con capacidad 1 y un cliente esperando, una segunda inserción se rechaza sin alterar la cola. Se conserva un solo cliente y se informa el sobreflujo.

Validaciones de entrada y operación

Calificación fuera de rango



La entrada 11 se rechaza porque la calificación debe estar entre 0 y 10. Los alumnos ya registrados mantienen sus notas y clasificación.

Verificación y entrega

Resultado comprobado: 70 verificaciones correctas, sin fallos, ejecutadas con Java 21.0.11. El registro completo se incluye en pruebas_resultados.txt.

Listas	Altas/retiros, rangos aleatorios, paridad y multiplicidad, umbral 7, orden alfabético, total exacto, claves e iniciales.
Pilas	LIFO, Unicode, palíndromos positivos/negativos, acarreos y ceros, 200 sumas grandes contra oráculo, reemplazo con conservación del orden.
Colas	FIFO, sobreflujo y vacío; espera de 18 s; llegadas, color y aceleración; fin con cinco coches; tarifa y mil nodos circulares.
Interfaz	Se activan los controles reales de los doce módulos; se revisan mensajes y datos. Los paneles se pintan a PNG mediante Swing en modo de prueba.

Origen de las imágenes. Son renderizaciones de los mismos paneles de la aplicación tras ejecutar las acciones de prueba. No son fotografías del monitor. Para las reglas temporales se usan instantes controlados; el uso normal toma el tiempo del sistema. Las capturas muestran resultados realmente producidos por el código.

Reproducción. Ejecutar COMPILAR_Y_PROBAR.cmd. El script compila, ejecuta pruebas, regenera imágenes y crea los cuatro JAR. Para actualizar este documento, ejecutar herramientas/generar_documentacion.py después de completar datos_portada.json; requiere Python con reportlab, Pillow y pypdf.

Archivos para entregar. El RAR incluye tres proyectos completos con fuentes y ejecutables, el menú general, este PDF, el enunciado original, las evidencias y el registro de pruebas. Verificar los nombres de integrantes y el grupo antes de subirlo.

Entrega indicada. Viernes 23 de octubre de 2026, por equipo; un integrante realiza la carga en Teams. La carga no se ha realizado desde este proyecto.

Fuente de requisitos. Instrumento F-SGC-033, revisión 00, Unidad II, Practica_U2_ED032026.pdf, 9 páginas, proporcionado por el usuario; copia de referencia en documentacion/Enunciado_original.pdf.

Trazabilidad de la implementación

Requisito	Clase o método
L1: productos	Estructuras.Almacen; ProductosPanel
L2: paridad	Estructuras.generarNumeros; ParesPanel
L3: alumnos	Estructuras.Grupo; AlumnosPanel
L4: compra	Estructuras.Compra; CompraPanel
L5: palabras	Estructuras.agrupar; PalabrasPanel
P1: invertir	Estructuras.invertir; InvertirPanel
P2: palíndromo	Estructuras.palindromo; PalindromoPanel
P3: suma	Estructuras.sumarGrandes; SumaPanel
P4: reemplazo	Estructuras.Reemplazar; ReemplazoPanel
C1: banco	Estructuras.Cola y Banco; BancoPanel
C2: juego	Estructuras.Juego; JuegoPanel y LienzoAutos
C3: estacionar	Estructuras.ColaCircularDoblementeLigada; ParkingPanel

Costos de las operaciones. Altas al final de ArrayList: O(1) amortizado; búsquedas por clave y clasificación de alumnos: O(n). Orden de compra: O(n log n). Invertir y sumar: O(n); reemplazar: O(n) y una pila auxiliar. Cola acotada: O(1) por inserción/extracción. En estacionamiento, enlazar o extraer es O(1); validar placas únicas recorre O(n).

Alcance de las pruebas. La evidencia verifica lógica y renderización Swing, sin depender de un IDE. La legibilidad final puede variar con la escala de pantalla del equipo; las ventanas son redimensionables y la tabla se puede desplazar. Los datos de trabajo, salvo el récord, no se guardan al cerrar.

Registro de pruebas automáticas

OK | L1: escanear 100 objetos Producto
OK | L1: rangos aleatorios de cantidad y precio
OK | L1: retiro sin pérdida del historial
OK | L1: no retirar dos veces
OK | L2: genera 500 números y clasifica todos correctamente
OK | L2: conserva todos los números, incluidos repetidos
OK | L2: rechaza cantidad fuera de rango
OK | L3: separación en el límite 7
OK | L3: reprobados extraídos de la lista inicial
OK | L3: rechaza notas mayores a diez
OK | L3: rechaza nombre vacío
OK | L4: orden alfabético por nombre
OK | L4: total decimal exacto 74.50
OK | L4: elimina por clave y recalcula total
OK | L4: claves únicas sin distinguir mayúsculas
OK | L4: clave inexistente
OK | L4: precio negativo
OK | L5: creación dinámica de listas e inicial acentuada
OK | L5: valida palabra individual
OK | P1: ejemplo UTM -> MTU
OK | P1: caracteres Unicode completos
OK | P1: entrada faltante
OK | P2: frase palíndroma
OK | P2: número y caso negativo
OK | P2: requiere letras o números
OK | P3: acarreo de 30 dígitos
OK | P3: cero y ceros iniciales
OK | P3: 200 sumas aleatorias de hasta 1234 dígitos contra oráculo BigInteger (sólo en pruebas)
OK | P3: contrato de enteros no negativos
OK | P3: rechaza letras
OK | P4: reemplaza todas las apariciones y conserva el orden
OK | P4: valor ausente
OK | P4: pila vacía
OK | C1: atiende cola vacía con mensaje
OK | C1: capacidad, frente y final

Registro de pruebas automáticas

OK | C1: rechaza sobreflujo
OK | C1: FIFO y tiempo de espera exacto de 18 segundos
OK | C1: queda vacía después de atender
OK | C2: primer coche e intervalo inicial de 20 segundos
OK | C2: no llega antes del tiempo
OK | C2: llegada exacta a los 20 segundos
OK | C2: color incorrecto no elimina el frente
OK | C2: la cola se conserva tras error
OK | C2: acelera de 20 a 18 segundos cada tres aciertos
OK | C2: récord con cantidad y tiempo
OK | C2: fin exacto en cinco coches y tiempo detenido en 80 s
OK | C2: no permite pintar después del final
OK | C2: reinicia partida y conserva récord
OK | C2: límite inferior de dos segundos
OK | C3: salida con estacionamiento vacío
OK | C3: enlaces anterior/siguiente y circularidad
OK | C3: rechaza placas duplicadas
OK | C3: salida FIFO y cobro de 30 x 2 = 60 pesos
OK | C3: elimina último nodo y restablece estructura
OK | C3: mil nodos sin cupo fijo; verifica FIFO y enlaces en cada salida
OK | Validación: separador decimal coma
OK | Validación: cantidad requiere entero
OK | Validación: desbordamiento numérico
OK | Validación: rechaza NaN
OK | Validación: datos faltantes
OK | Interfaz: doce pantallas y acciones ejecutadas mediante sus controles
OK | Interfaz L1: botón de retiro actualiza datos y tabla
OK | Interfaz L2: botón genera las tres listas
OK | Interfaz C1: botón captura sobreflujo sin modificar la cola
OK | Interfaz P3: mensaje adecuado al introducir letras
OK | Interfaz L3: muestra error de calificación fuera de rango
OK | Interfaz L4: muestra error de datos faltantes
OK | Proyecto independiente Listas: menú con 5 ejercicios
OK | Proyecto independiente Pilas: menú con 4 ejercicios
OK | Proyecto independiente Colas: menú con 3 ejercicios

TOTAL: 70 verificaciones correctas. Sin fallos.